

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



ANALAYSIS AND DESIGN OF ALGORITHMS AAT REPORT on

CODING CHALLENGES

Submitted by

Khushi B Chilakawad (1BF24CS137)
Madhav V Nair (1BF24CS159)

Under the Guidance of

MANJULA S

Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

February to June 2026

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, **Khushi B Chilakawad (1BF24CS137)** and **Madhav V Nair (1BF24CS159)**, students of 4th Semester F Section, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this Coding Challenges for the course Analysis and Design of Algorithms(23CS4PCADA) has been carried out by us under the guidance of MANJULA

S, Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester February to June 2026.

I also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

Khushi B Chilakawad (1BF24CS137)
Madhav V Nair (1BF24CS159)

Coding challenge 1: Palindrome Partitioning

Given a string *s*, partition *s* such that every substring of the partition is a palindrome.
Return all possible palindrome partitioning of *s*.

Example 1:

Input: *s* = "aab"

Output: [["a","a","b"],["aa","b"]]

Example 2:

Input: *s* = "a"

Output: [["a"]]

Constraints:

- $1 \leq s.length \leq 16$
- *s* contains only lowercase English letters.

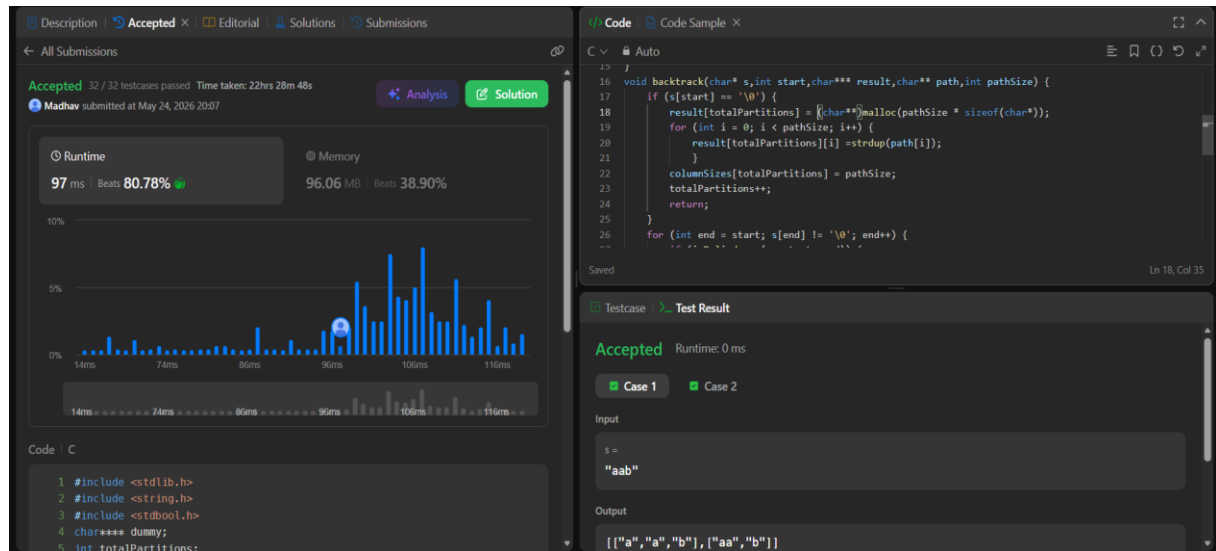
Solution:

Design Technique chosen with justification:

Backtracking

Backtracking is suitable because the problem requires generating all possible palindrome partitions of a string. At each step, the algorithm tries every possible substring starting from the current index, checks whether it is a palindrome, and recursively continues partitioning the remaining string. If a chosen substring does not lead to a valid complete partition, the algorithm backtracks and tries another possibility. This follows the classic backtracking approach of choose, recurse, and undo, making it effective for exploring all valid combinations.

Screenshots of the output from LeetCode platform:



Coding challenge 2: 315. Count of Smaller Numbers After Self

Given an integer array `nums`, return an integer array `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`.

Example 1:

Input: `nums = [5,2,6,1]`

Output: `[2,1,1,0]`

Explanation:

To the right of 5 there are 2 smaller elements (2 and 1). To the right of 2 there is only 1 smaller element (1).

To the right of 6 there is 1 smaller element (1). To the right of 1 there is 0 smaller element.

Example 2:

Input: `nums = [-1]`

Output: `[0]`

Example 3:

Input: `nums = [-1,-1]`

Output: `[0,0]`

Constraints:

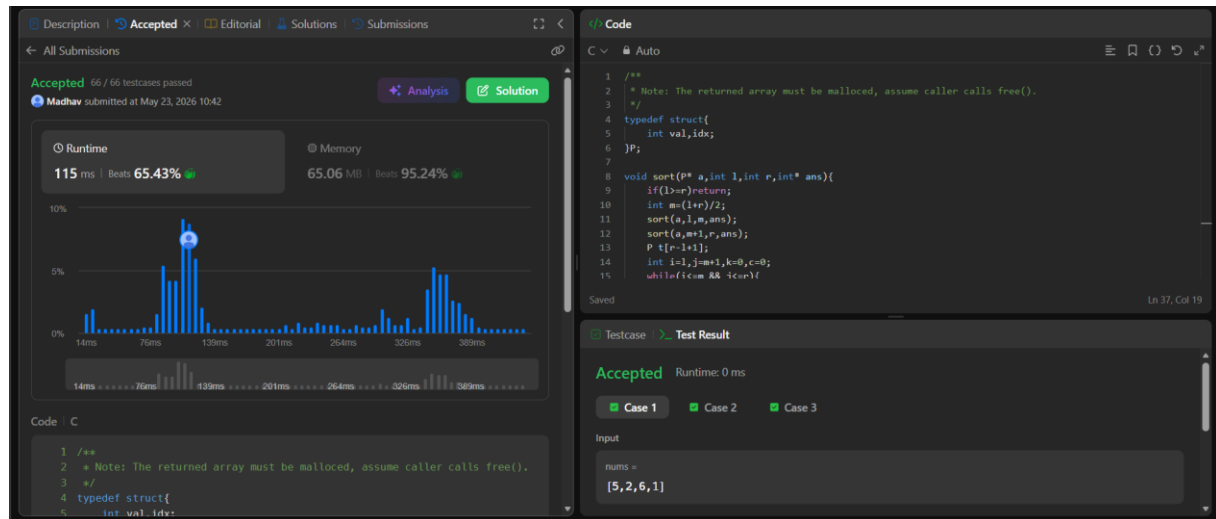
- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Solution:

Design Technique chosen with justification:

The design technique used in this solution is Modified Merge Sort, which is based on the Divide and Conquer approach. The array is recursively divided into smaller halves until single elements remain, and during the merge step, the algorithm counts how many smaller elements from the right half move before elements in the left half. These counts are added to the answer array using the original indices stored in the structure. By combining sorting and counting during merging, the algorithm efficiently reduces the time complexity from brute force $O(n^2)$ to $O(n \log n)$.

Screenshots of the output from LeetCode platform:



Coding challenge 3: 295. Find Median from Data Stream

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $arr = [2,3,4]$, the median is 3.
- For example, for $arr = [2,3]$, the median is $(2 + 3) / 2 =$

2.5. Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum",  
"findMedian"] [[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();  
medianFinder.addNum(1);    // arr = [1]  
medianFinder.addNum(2);    // arr = [1, 2]  
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)  
medianFinder.addNum(3);    // arr[1, 2, 3]  
medianFinder.findMedian(); // return 2.0
```

Constraints:

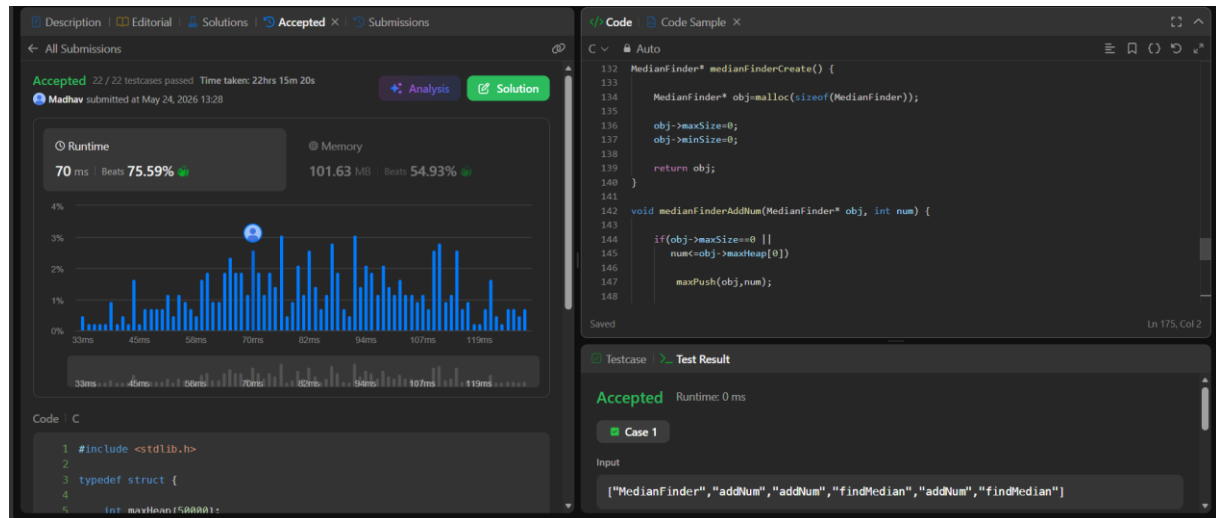
- $-10^5 \leq num \leq 10^5$
- There will be at least one element in the data structure before calling findMedian.
- At most $5 * 10^4$ calls will be made to addNum and findMedian.

Solution:

Design Technique chosen with justification:

The design technique used in this solution is the Heap (Priority Queue) technique using two heaps: a Max Heap for the smaller half of the numbers and a Min Heap for the larger half. When a new number is added, the heaps are balanced so that their sizes differ by at most one. This allows the median to be found efficiently using the top elements of the heaps instead of sorting the entire array repeatedly. The approach optimizes insertion to $O(\log n)$ and median retrieval to $O(1)$, making it much faster than sorting-based methods.

Screenshots of the output from LeetCode platform:



Coding challenge 4: 435. Non-overlapping Intervals

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping. Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`

Output: 2

Explanation: You need to remove two `[1,2]` to make the rest of the intervals non-overlapping.

Example 3:

Input: `intervals = [[1,2],[2,3]]`

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 10^5$
- `intervals[i].length == 2`
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

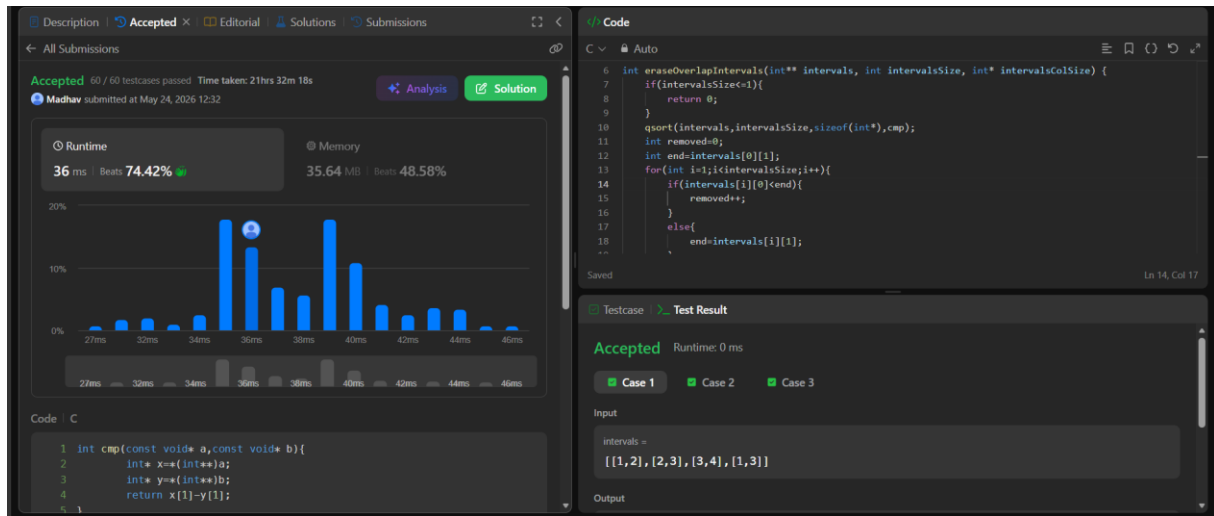
Solution:

Design Technique chosen with justification:

Greedy Algorithm

The design technique used in this solution is the Greedy Algorithm. The intervals are first sorted based on their ending times using `qsort()`. The algorithm then greedily keeps the interval with the smallest end value because it leaves maximum space for upcoming intervals and minimizes overlaps. During traversal, if the current interval starts before the end of the previously selected interval, it is considered overlapping and removed. Otherwise, the interval is kept by updating the end value. This greedy strategy efficiently solves the problem with overall time complexity $O(n \log n)$ due to sorting.

Screenshots of the output from LeetCode platform:

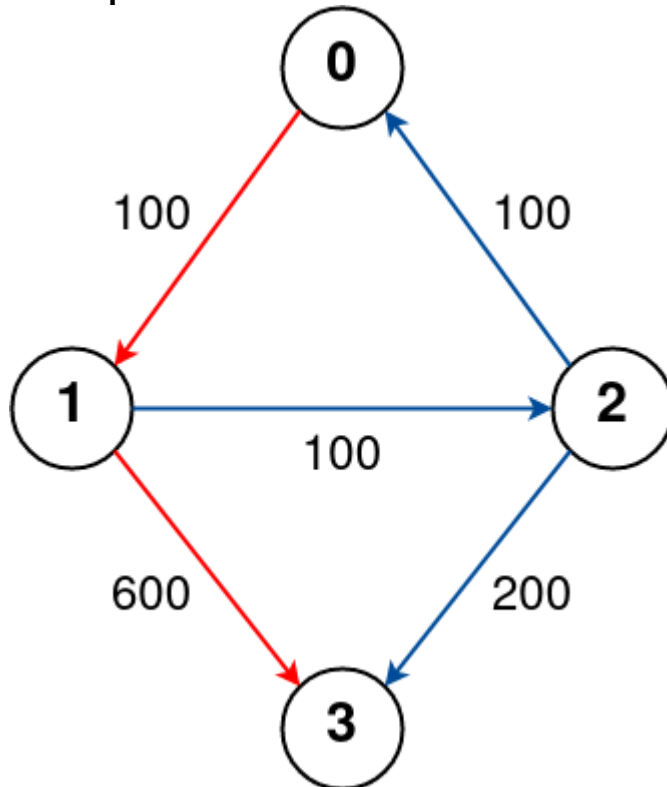


Coding challenge 5: 787. Cheapest Flights Within K Stops

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Example 1:



Input: $n = 4$, `flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]`, `src = 0`, `dst = 3`, `k = 1`

Output: 700

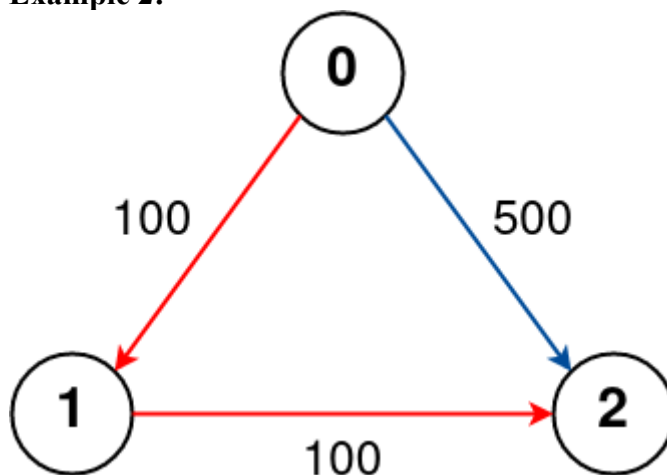
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities `[0,1,2,3]` is cheaper but is invalid because it uses 2 stops.

Example 2:



Input: $n = 3$, $\text{flights} = [[0,1,100],[1,2,100],[0,2,500]]$, $\text{src} = 0$, $\text{dst} = 2$, $k = 1$

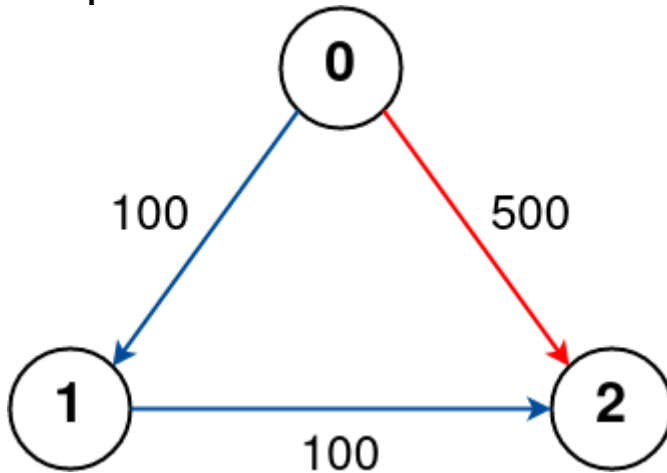
Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost $100 + 100 = 200$.

Example 3:



Input: $n = 3$, $\text{flights} = [[0,1,100],[1,2,100],[0,2,500]]$, $\text{src} = 0$, $\text{dst} = 2$, $k = 0$

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $2 \leq n \leq 100$
- $0 \leq \text{flights.length} \leq (n * (n - 1) / 2)$
- $\text{flights}[i].\text{length} == 3$
- $0 \leq \text{from}_i, \text{to}_i < n$
- $\text{from}_i \neq \text{to}_i$
- $1 \leq \text{price}_i \leq 10^4$
- There will not be any multiple flights between two cities.
- $0 \leq \text{src}, \text{dst}, k < n$
- $\text{src} \neq \text{dst}$

Solution:

Design Technique chosen with justification:

Bellman-Ford Algorithm (or Breadth-First Search / Relaxation)

If we try a standard Dijkstra's algorithm, it might find a cheaper path that uses too many stops and exceeds the limit K , because it always focuses on the absolute shortest distance first.

Instead, we use a modified Bellman-Ford algorithm that loops exactly $K + 1$ times. In each loop, we create a copy of our prices array from the previous step. We then look at every flight and try to lower the cost to reach each city. By using the old prices to calculate the new ones, we make sure we only extend our path by exactly one extra stop per loop. This guarantees we find the cheapest price without ever crossing the limit K .

Screenshots of the output from LeetCode platform:

